

SIT225 Data Capture Technologies Assessment

Pass Task 1

YAT LONG TSE



Certificate of Completion

General Makerspace Induction
Monday, July 27, 2026

SIT MakerSpace

Part A: Use case & problem statement

Many people have to share spaces like school lockers, college dorm room or offices, which makes it easy for others to touch, move, or steal their personal items like laptops, books and money. How can we build a cheap and easy tool that catches someone trying to touch or steal your personal things? This project looks at how to quickly warn an owner with lights, sounds or phone messages the moment someone gets too close to their stuff.

This matters to anyone who shares a room or office like students in dorms or workers at shared desks, who worry about people touching their laptops, money or books. It lets other people know immediately by scaring them away with loud noises and bright lights, and it lets the owner know right away so they can run back and check on their things.

I also added accelerometer sensor so it can detect device movement if someone move the device it might lead to motion sensor not working correctly.

The project will show that we can use motion sensor and different electronic parts to create a safe zone around your locker or desk. I expect it to have light and sound, I will try to make the message function work if Arduino cloud let me do it.

Part B: Sensor & Protocol Design

Sensor(s) selected

PIR motion sensor:

Why this sensor?

It can be the core part of my device, it can detect human body heat moving around. When placed, it knows if a person is standing or walking in the area it detects.

Sensor Specifications:

Range: Up to 7 meters

Accuracy: High for picking up large movements like a human walking, but it can be affected by sudden hot or cold air.

Output Format: Digital signal, it outputs a HIGH/1 when it sees motion and a LOW/0 when everything is still.

Accelerometer

Why this sensor?

It can detect if the whole device is being moved, by adding this sensor now it has the function to prevent other people from moving the whole device to somewhere else without notice.

Sensor Specifications:

Range: Measures acceleration up to 2g, 4g, 8g, or 16g (forces of gravity).

Accuracy: Very high; it can pick up tiny tilts and vibrations.

Output Format: Digital data through standard communication pins, sending exact numbers for three directions (X , Y , and Z axes).

2. Data capture setup

Hardware	Arduino board + sensor + USB connection to laptop
Communication	Serial port (COM3)
Sampling frequency	PIR & Accelerometer: 5 times per second
Duration	30 minutes per section
Timestamp	Python Datatime

Hardware Components: An Arduino board connected to three sensors (PIR Motion Sensor, Accelerometer, and DHT22 Temperature/Humidity Sensor). The Arduino is plugged directly into a laptop using a standard USB connection to power the device and transmit data. And they will be using bread board to connect.

Communication: Data is sent from the Arduino to the laptop via the Serial Port COM3 on Windows. A Python script running on the laptop reads this serial data to save it automatically.

Sampling Frequency:

PIR & Accelerometer: Checked every 5 times per second because human movement and device theft happen instantly.

Duration: The data will be collected for 30 minutes for each test. 30 minutes will be enough data for check if device work or not.

Timestamping: I will be using Python datetime for timestamps so it easier to read.

3. Local processing & storage

- Data stream: Arduino sends timestamp (optional), sensor_value pairs via serial
- Python script (or similar) receives and buffers data, (and generates timestamp, if not sent from Arduino)
- Storage: Save to CSV file

4. Data quality checks

missing or invalid readings: if reading return nan or not number I will just remove them.

Outliers: I will use Threshold Filtering to make a boundary rules based on physical reality to catch and remove these errors.

Accelerometer: force > 16g , delete or ignore

PIR Sensor: values not 0 or 1 to go reject value.

Data Filtering and Smoothing: for accelerometer is highly sensitive so I might need to add some Moving Average Filter or lower limit value.

Part C: Implementation & Data Capture

Arduino .ino sketch(security_device.ino)

```
// Add sensor library
```

```
#include <Arduino_LSM6DS3.h>
```

```
// PIN setup
```

```
#define PIR_PIN 2          // PIR sensor signal connected to Pin
```

```
#define RED_LED_PIN 4      // LED
```

```
// data filtering (Accelerometer Moving Average) ---
```

```
const int numReadings = 5;          // Smooth across 5 readings
```

```
float readingsX[numReadings];
```

```
float readingsY[numReadings];
```

```
float readingsZ[numReadings];
```

```
int readIndex = 0;                  // Index of current reading
```

```
float totalX = 0, totalY = 0, totalZ = 0;
```

```
float avgX = 0, avgY = 0, avgZ = 0;
```

```
// ACCELEROMETER ALARM THRESHOLD
```

```
const float movementThreshold = 0.3;
```

```
void setup() {  
    // Initialize Serial communication at 9600 baud rate  
    Serial.begin(9600);  
  
    // Init PIR Sensor  
    pinMode(PIR_PIN, INPUT);  
    pinMode(RED_LED_PIN, OUTPUT);  
  
    // Init Accelerometer  
    if (!IMU.begin()) {  
        Serial.println("Error: LSM6DS3 IMU accelerometer not detected!");  
        while (1);  
    }  
  
    // Initialize all moving average array readings to 0  
    for (int thisReading = 0; thisReading < numReadings; thisReading++) {  
        readingsX[thisReading] = 0;  
        readingsY[thisReading] = 0;  
        readingsZ[thisReading] = 0;  
    }  
}  
  
void loop() {  
    // 1. Read PIR Sensor (Outputs 0 or 1)  
    int pirValue = digitalRead(PIR_PIN);
```

```
// 2. Read LSM6DS3 Accelerometer data
```

```
float rawX = 0, rawY = 0, rawZ = 0;
```

```
if (IMU.accelerationAvailable()) {
```

```
    IMU.readAcceleration(rawX, rawY, rawZ);
```

```
}
```

```
// 3. Data Quality Check: Filter out accelerometer anomalies > 16g
```

```
if (abs(rawX) <= 16.0 && abs(rawY) <= 16.0 && abs(rawZ) <= 16.0) {
```

```
    // Subtract the oldest reading from running total
```

```
    totalX -= readingsX[readIndex];
```

```
    totalY -= readingsY[readIndex];
```

```
    totalZ -= readingsZ[readIndex];
```

```
    // Read new values into the array
```

```
    readingsX[readIndex] = rawX;
```

```
    readingsY[readIndex] = rawY;
```

```
    readingsZ[readIndex] = rawZ;
```

```
    // Add new reading to running total
```

```
    totalX += readingsX[readIndex];
```

```
    totalY += readingsY[readIndex];
```

```
    totalZ += readingsZ[readIndex];
```

```
    // Advance to next position in array
```

```
    readIndex++;

    if (readIndex >= numReadings) {
        readIndex = 0;
    }

    // Calculate final smoothed averages (in g)

    avgX = totalX / numReadings;
    avgY = totalY / numReadings;
    avgZ = totalZ / numReadings;
}

// 4. Check Alarm Conditions

// Calculate current orientation differs from rest.

bool motionDetected = (pirValue == 1);

bool forceDetected = (abs(avgX) > movementThreshold ||
                      abs(avgY) > movementThreshold ||
                      abs(abs(avgZ) - 1.0) > movementThreshold);

if (motionDetected || forceDetected) {
    digitalWrite(RED_LED_PIN, HIGH);
} else {
    digitalWrite(RED_LED_PIN, LOW);
}

// 5. Print Data to Serial (Processed by Python)

// Format: PIR,AccelX,AccelY,AccelZ

Serial.print(pirValue);
```

Gordon Tse (s223532749)

```
Serial.print("");  
Serial.print(avgX, 3);  
Serial.print(",");  
Serial.print(avgY, 3);  
Serial.print(",");  
Serial.println(avgZ, 3);  
  
// 5. 200ms delay to make the loop run 5 times per sec  
delay(200);  
}
```

Python script (data_capture.py)

```
import serial

import csv

import time

from datetime import datetime


ser = serial.Serial('COM3', 9600, timeout=1)


time.sleep(2)


try:
    with open('my_data.csv', 'w', newline='') as f:
        writer = csv.writer(f)

        # Create clear headers for all your sensor outputs
        writer.writerow(['timestamp', 'PIR_Motion', 'Accel_X', 'Accel_Y', 'Accel_Z'])

    while True:

        # Read a line of data from the Arduino
        line = ser.readline().decode('utf-8').strip()

        if line: # Only process if we actually got data (ignores empty
timeouts)

            timestamp = datetime.now().strftime('%Y-%m-%d
%H:%M:%S.%f')[:-3] # Friendly YYYY-MM-DD HH:MM:SS.ms format


            # Split the comma-separated line into individual values
```

```
        sensor_values = line.split(',')

        # Ensure we got all 4 expected values from the Arduino before
saving
        if len(sensor_values) == 4:

            writer.writerow([timestamp] + sensor_values)

            f.flush() # Forces Python to save the data immediately so
you don't lose anything

except KeyboardInterrupt:

    print("\nData collection stopped by user. CSV file saved successfully!")

finally:

    ser.close() # Safely close the serial port
```

Python script (data_analysis.ipynb)

```
# %% [markdown]

# ## 1. Load Data

# %%

import pandas as pd

import matplotlib.pyplot as plt


# %%

try:

    df = pd.read_csv('my_data.csv')

    print("CSV successfully loaded into Pandas DataFrame.\n")

except FileNotFoundError:

    print("Error: 'my_data.csv' not found. Run your data collection script first!")

    exit()


# %%

# Print first few rows to confirm

df.columns = df.columns.str.strip()

print("First 3 rows of data:")

print(df.head(3), "\n")


# %%

# Convert timestamp to datetime objects

df['timestamp'] = pd.to_timedelta('00:' + df['timestamp'].astype(str))

df['elapsed_seconds'] = df['timestamp'].dt.total_seconds()
```

Gordon Tse (s223532749)

```
# %%
```

```
print("First 3 rows of data:")
```

```
print(df.head(3), "\n")
```

```
# %%
```

```
# Check for data quality issues
```

```
print("--- Data Quality Checks ---")
```

```
print(f"Total rows collected: {len(df)}")
```

```
print(f"Missing values per column:\n{df.isnull().sum()}\n")
```

```
print(f"Duplicate rows found: {df.duplicated().sum()}\n")
```

```
print("Data types after conversion:")
```

```
print(df.dtypes, "\n")
```

```
# %%
```

```
# Report statistical summary (Mean, Min, Max, Std Dev)
```

```
print("--- Basic Sensor Statistics ---")
```

```
stats = df[['PIR_Motion', 'Accel_X', 'Accel_Y', 'Accel_Z']].describe()
```

```
# Reordering/filtering to display specifically requested stats
```

```
requested_stats = stats.loc[['mean', 'min', 'max', 'std']]
```

```
print(requested_stats.to_string())
```

```
print("\n")
```

```
# %% [markdown]
```

```
# ## 2. Create Visualization
```

```
# %%
```

```
# 1. Identify and mask physics-defying hardware spikes (> 5g or < -5g)
```

```
for col in ['Accel_X', 'Accel_Y', 'Accel_Z']:

    # Replace outliers with NaN

    df[f'{col}_Clean'] = df[col].mask((df[col] > 3.0) | (df[col] < -3.0))

    # Forward-fill and backward-fill the removed outlier points

    df[f'{col}_Clean'] = df[f'{col}_Clean'].ffill().bfill()

# 2. Apply smoothing to the cleaned data

df['PIR_Smooth'] = df['PIR_Motion'].rolling(window=30, center=True).mean()

df['Accel_X_Clean'] = df['Accel_X_Clean'].rolling(window=3, center=True).mean()

df['Accel_Y_Clean'] = df['Accel_Y_Clean'].rolling(window=3, center=True).mean()

df['Accel_Z_Clean'] = df['Accel_Z_Clean'].rolling(window=3, center=True).mean()

# %%

plt.figure(figsize=(12, 5))

# Plot cleaned signals instead of raw unmasked data

plt.plot(df['elapsed_seconds'], df['Accel_X_Clean'], label='Accel X
(Clean)',alpha=0.8)

plt.plot(df['elapsed_seconds'], df['Accel_Y_Clean'], label='Accel Y
(Clean)',alpha=0.8)

plt.plot(df['elapsed_seconds'], df['Accel_Z_Clean'], label='Accel Z
(Clean)',alpha=0.8)

# Set realistic y-limits for accelerometer values (gravity ~1.0g)

plt.ylim(-0.4, 1.4)

plt.xlabel('Elapsed Seconds')
```

Gordon Tse (s223532749)

```
plt.ylabel('Acceleration (g)')
```

```
plt.title('Cleaned Accelerometer Readings (Zoomed)')
```

```
plt.legend(loc='upper right')
```

```
plt.grid(True)
```

```
plt.savefig('accelerometer_plot.png', dpi=300)
```

```
print("Plot generated successfully and saved locally as  
'accelerometer_plot.png'!")
```

```
plt.show()
```

```
# %%
```

```
fig, ax = plt.subplots(figsize=(10, 5))
```

```
# -----
```

```
# Time vs. PIR Motion
```

```
# -----
```

```
# Smooth data
```

```
ax.plot(df['elapsed_seconds'], df['PIR_Smooth'], label='Motion Intensity',  
color='crimson', linewidth=2)
```

```
ax.fill_between(df['elapsed_seconds'], df['PIR_Smooth'], color='crimson',  
alpha=0.15)
```

```
# Chart Styling
```

```
ax.set_title('PIR Motion Detection Frequency & Intensity Trend', fontsize=12,  
fontweight='bold')
```

```
ax.set_xlabel('Elapsed Time (Seconds)', fontsize=10)
```

Gordon Tse (s223532749)

```
ax.set_ylabel('Activity Rate', fontsize=10)
```

```
ax.set_ylim(-0.05, 1.05)
```

```
ax.grid(True, linestyle='--', alpha=0.5)
```

```
ax.legend(loc='upper right')
```

```
plt.tight_layout()
```

```
# Save and show
```

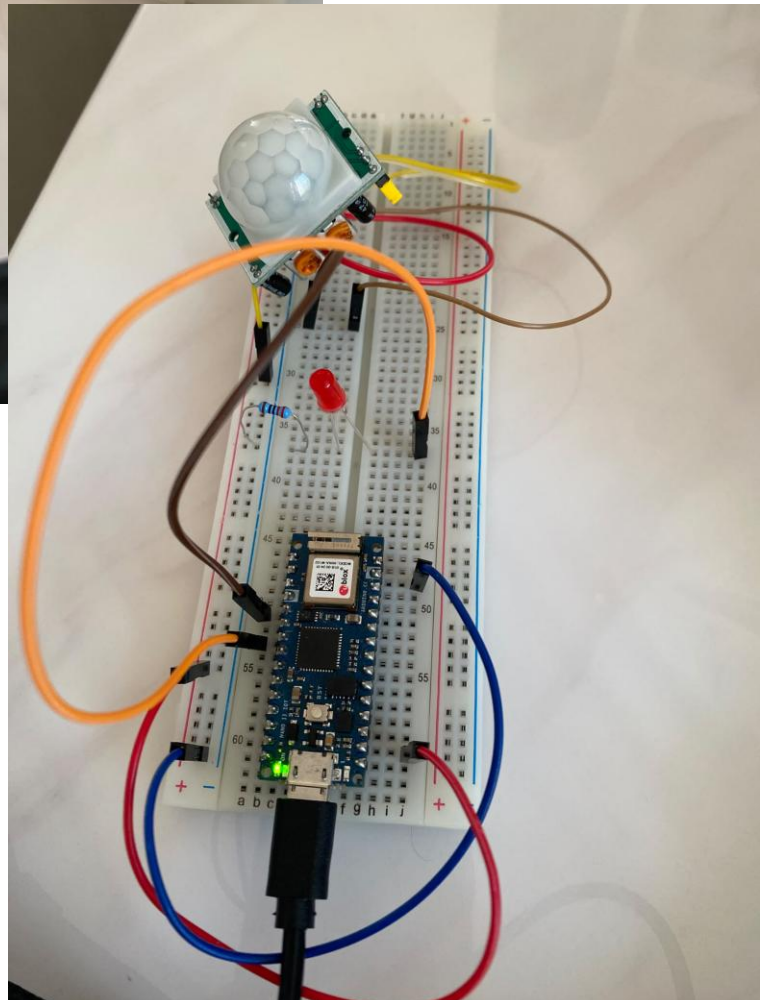
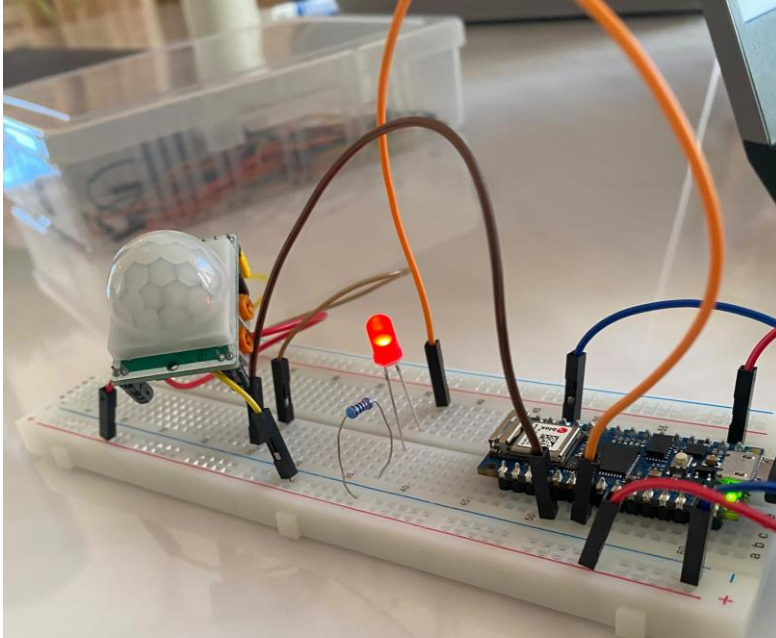
```
plt.savefig('pir_motion_plot.png', dpi=300)
```

```
print("Plot generated successfully and saved locally as 'pir_motion_plot.png'!")
```

```
plt.show()
```

Gordon Tse (s223532749)

Hardware Setup



Arduino .ino sketch expiation

1. Variable Setup

- i. Add sensor library: the code will need Arduino_LSM6DS3.h library to work for Accelerometer
- ii. Pin set up: I let pin D2 for PIR sensor data link and pin 4 to LED
- iii. Data Smoothing Setup: set up array and variable for later data smoothing use

2. Setup() function:

- i. Serial.begin(9600) Opens serial communication at 9600 baud rate
- ii. PinMode to set pin input and output.
- iii. !IMU.begin() use for if the arduino board not connect then print error message then continue until it connect.
- iv. Initialize all moving average array readings to 0

3. Loop()

- i. read PIR sensor (getting 0 or 1)
- ii. read Accelerometer (getting x,y,z)
- iii. filter out Accelerometer reading if it > 16g because it might be error to get the large G. After filtering I do calculate moving average for accelerometer. In short: Subtract the oldest reading from running total, Read new values into the array, add new reading to running total. And $\text{total} / \text{NumberReading} = \text{MA}$.

The reason I add moving average here is Accelerometer is sensitive even some large sound or big movement from very far will detect by Accelerometer, therefore I add MA to make it less sensitive.

- iv. LED light up conditions, I set it to if Accelerometer or PIR sensor detect someone is here then LED will light.
- v. Print data to serial monitor
- vi. Delay(200) to let loop run 5 times per sec.

data_capture.py sketch expiation

1. Add library:

- i. serial for board connection
- ii. csv for record data in csv file
- iii. time and datetime for managing the time in csv

2. Setup:

- i. Connect board with “COM3” at 9600 rate
- ii. Stop program for 2 sec to let it connect

3. Main program:

- i. Use try and except to stop the program
- ii. Open or create my_data.csv in “write mode”
- iii. Write header 'timestamp', 'PIR_Motion', 'Accel_X', 'Accel_Y', 'Accel_Z' to csv file.
- iv. Keep reading data from Arduino board
- v. If successful get back data, timestamp to YYYY-MM-DD HH:MM:SS.ms format
- vi. Use split(“,”) to get individual value.
- vii. If after split is 4 individual value('PIR_Motion', 'Accel_X', 'Accel_Y', 'Accel_Z') then write value to csv file.also the timestamp.
- viii. Save csv file.

CSV File Data

自動儲存 關閉

my_data.csv • 已儲存到此電腦

檔案 常用 插入 繪圖 頁面配置 公式 資料 校閱 檢視 自動化 說明

剪下 貼上 複製 複製格式 剪貼簿

新細明體 12 A⁺ A⁻ B I U 字型 對齊方式

A1	:	×	✓	fx	timestamp			
	A	B	C	D	E	F	G	H
1	timestamp	PIR_Motion	Accel_X	Accel_Y	Accel_Z			
2	54:24.4	0	0.024	0.007	1.018			
3	54:24.4	0	0.024	0.007	1.017			
4	54:24.4	0	0.024	0.007	1.017			
5	54:24.4	0	0.024	0.007	1.016			
6	54:24.4	0	0.025	0.007	1.016			
7	54:24.4	0	0.025	0.007	1.016			
8	54:24.4	0	0.024	0.007	1.017			
9	54:24.4	0	0.024	0.006	1.017			
10	54:24.4	0	0.024	0.007	1.017			
11	54:24.4	0	0.024	0.007	1.017			
12	54:24.5	0	0.024	0.006	1.017			
13	54:24.7	0	0.024	0.007	1.017			
14	54:24.9	0	0.024	0.006	1.017			
15	54:25.1	0	0.024	0.005	1.018			
16	54:25.3	1	0.024	0.005	1.017			
17	54:25.5	1	0.024	0.005	1.017			
18	54:25.7	1	0.024	0.006	1.017			
19	54:25.9	1	0.024	0.006	1.017			
20	54:26.1	1	0.024	0.007	1.017			
21	54:26.3	0	0.024	0.007	1.017			
22	54:26.5	0	0.024	0.007	1.017			
23	54:26.7	0	0.025	0.006	1.015			
24	54:27.0	0	0.025	0.006	1.015			
25	54:27.2	0	0.024	0.005	1.017			
26	54:27.4	0	0.024	0.005	1.016			
27	54:27.6	0	0.025	0.004	1.016			
28	54:27.8	0	0.024	0.004	1.018			
29	54:28.0	0	0.024	0.005	1.019			
30	54:28.2	0	0.024	0.004	1.017			
31	54:28.4	0	0.024	0.003	1.017			
32	54:28.6	0	0.024	0.004	1.017			
33	54:28.8	0	0.023	0.003	1.019			
34	54:29.0	0	0.024	0.002	1.017			
35	54:29.2	0	0.025	0.002	1.017			
36	54:29.4	0	0.024	0.003	1.018			
37	54:29.6	0	0.024	0.003	1.017			
38	54:29.8	0	0.024	0.003	1.017			
39	54:30.0	1	0.024	0.003	1.017			
40	54:30.2	1	0.024	0.003	1.017			
41	54:30.4	1	0.024	0.003	1.016			
42	54:30.6	1	0.024	0.003	1.017			
43	54:30.8	1	0.024	0.003	1.017			
44	54:31.0	1	0.024	0.003	1.017			
45	54:31.2	0	0.024	0.003	1.017			
46	54:31.4	0	0.024	0.003	1.017			
47	54:31.6	0	0.024	0.003	1.016			
48	54:31.8	0	0.024	0.003	1.017			
49	54:32.0	0	0.024	0.003	1.017			
50	54:32.2	0	0.024	0.003	1.017			

< >

my_data

+

IDE code and Serial Monitor

 **Arduino NANO 33 IoT** ▼

security_device.ino

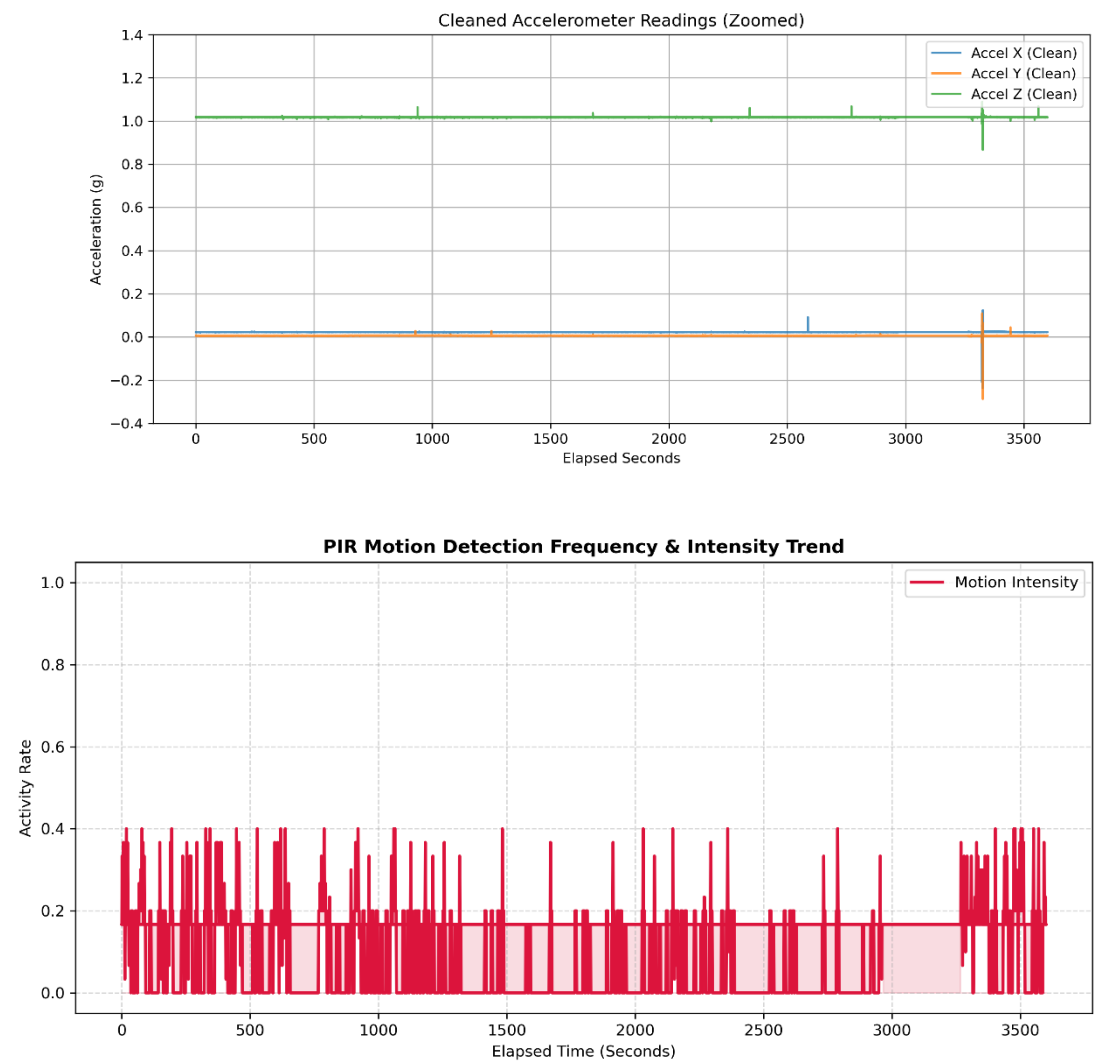
```
84 |   avgZ = totalZ / numReadings;
85 | }
86 |
87 | // 4. Check Alarm Conditions
88 | // Calculate current orientation differs from rest.
89 | bool motionDetected = (pirValue == 1);
90 | bool forceDetected = (abs(avgX) > movementThreshold ||
91 |                      | abs(avgY) > movementThreshold ||
92 |                      | abs(abs(avgZ) - 1.0) > movementThreshold);
93 |
94 | if (motionDetected || forceDetected) {
95 |   digitalWrite(RED_LED_PIN, HIGH);
96 | } else {
97 |   digitalWrite(RED_LED_PIN, LOW);
98 | }
```

Output Serial Monitor ✕

Message (Enter to send message to 'Arduino NANO 33 IoT' on 'COM3')

0,0.021,0.003,1.024
0,0.021,0.004,1.023
0,0.021,0.003,1.022
0,0.021,0.003,1.022
0,0.021,0.004,1.024
0,0.020,0.006,1.030
0,0.020,0.006,1.031
0,0.021,0.006,1.030
0,0.020,0.006,1.030
0,0.020,0.006,1.030
0,0.021,0.004,1.021
0,0.022,0.004,1.020
0,0.021,0.005,1.023
0,0.021,0.005,1.023
0,0.021,0.004,1.021
0,0.021,0.004,1.021
0,0.020,0.005,1.023
0,0.021,0.004,1.022
0,0.021,0.004,1.020
0,0.022,0.004,1.020
0,0.022,0.003,1.020

Part D: Analysis & Visualisation



For the data 'timestamp', 'PIR_Motion', 'Accel_X', 'Accel_Y', 'Accel_Z' connected in CSV, I have made a python code to analysis and made a chart to visualize the data for better understanding what happened during the data.

First of all, I want to see sensor statistics with mean, min, max, std.

Sensor Statistics

	PIR_Motion	Accel_X	Accel_Y	Accel_Z
mean	0.076663	0.026686	0.007212	1.078577

min	0.000000	-0.307000	-0.350000	0.826000
max	1.000000	23.000000	8.000000	1016.000000
std	0.266064	0.302980	0.062957	7.955093

For the Statistics most value are quite accurate however I can see Accl_Z max have the value 1016 which is a error data capture. There are nothing to worry about I already do some filtering in the chart code but it still good to know there are error reading here.

For the first chart I want to make a chart about accelerometer detected force over time. I use smoothing data 'Accel_X', 'Accel_Y', 'Accel_Z' and plot them with 3 different line. Blue is X, orange is Y and green is Z. I have set the force to -0.4 to 1.4 which is the range that our accelerometer will output. Higher or lower than that might be an error. For time I set to seconds. Look at the chart I can see blue and orange line mostly stay at 0g and green stay at 1g because green is the gravity as it always stay at 1g if it no one touch and stay on earth.

At around 2600s and 3400s, I can see blue(X) have a increase of force. That mean someone move it. same as orange(Y) it move increase at around 3400s and 3500s. Looking at green(Z) the increase happened at 900s, 1700s, 2400s, 2800s, 3300s and 3500s. it all showing that someone pick it up at these time. By looking at this chart I can see when people move my device.

For second chart I want see when my PIR sensor detect people. So I made chart PIR motion detected VS time. On the Y axis is activity rate and X axis is time in seconds. When I making this chart I find out that is I use raw data in the chart it will be to much signal and it become messy so I use 30 row smooths to show less signal. I can confirm that the data is correct because I put my deceive in my living room when capturing data. Therefore lot of people pass the device. For this chart I use PIR motion data.

Looking at this chart red line represents the motion. I can see there are so many activity going on during the time 0s-1500s, that mean people are moving around my area. When it come to 1500s – 3000s, there are less activity going on, less

people are moving around. During 3000s – 3250s there are no activity so I can see no people near my device. 3250s – 3600s people are near the device again.

By combining the two chart, I can tell when actually someone touch or move my device and in my area to confirm that someone is here for sure. Like at 2600s blue(X) increase and motion are detected it tell us someone in area and has move device. By use this method I can support my problem statement where it can let user know when people enter the area and when people move your item.

Of Couse there are limitation like I cannot track who move the device or who enter the room, or like what we see in motion chart it might get lot of signal when lot of people in area.

For future improvements I want to add one more senser like DTH22 sensor to track the temperature or humidity to see if the environment will damage the item I try to protect. Also If possible I want to use cloud and once detect people it can send message to my phone.

"This assessment work was developed with assistance from the following approved Deakin genAI tools:

- **Google Gemini**

Used to All genAI-generated suggestions were critically evaluated and integrated only where supported by evidence. The final design, decisions, and justifications are my own."

prompt

"i am running IDE and try to run py file. if IDE running and py not work how py gonna record the data?"

Output

"You can have the Arduino IDE window open while running your Python script. The only thing you cannot do is open the **Serial Monitor or **Serial Plotter** windows inside the IDE at the same time."**

My explanations

After reading this I know I know I need to close IDE first then run my py file or else it will get block.

prompt

"how can i save data in csv file using py?"

Output

The easiest way to save data to a CSV file in Python is using the built-in csv module, which requires no extra installations.

Here is the standard, most reliable way using Python's native csv module and a with statement (which automatically opens and closes the file safely):

Python

Gordon Tse (s223532749)

```
import csv
```

```
# Your data as a list of rows
```

```
data = [  
    ["Name", "Age", "City"],  
    ["Alice", 28, "Melbourne"],  
    ["Bob", 35, "Sydney"],  
    ["Charlie", 22, "Brisbane"]  
]
```

```
# Write to CSV
```

```
with open("output.csv", mode="w", newline="", encoding="utf-8") as file:
```

```
    writer = csv.writer(file)
```

```
    writer.writerows(data)
```

My explanations

I used the code

```
with open("output.csv", mode="w", newline="", encoding="utf-8") as file: writer =  
csv.writer(file) writer.writerows(data)
```

on my project py code to write my data to it.

prompt

“how can i make sub plot chart ?”

Output

The standard and most flexible way to create subplots in Python is using **Matplotlib** with `plt.subplots()`. It creates a grid of plots (called axes) inside a single figure.

Here are the most common ways to use it.

1. Basic 1x2 Side-by-Side Plots

When you create a single row of plots, axes works like a 1D list (`axes[0]`, `axes[1]`).

```
import matplotlib.pyplot as plt

# Sample data
x = [1, 2, 3, 4, 5]
y1 = [2, 4, 6, 8, 10]
y2 = [1, 4, 9, 16, 25]

# Create 1 row, 2 columns of subplots
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))

# First subplot (left)
axes[0].plot(x, y1, color="blue", marker="o")
axes[0].set_title("Linear Growth")
axes[0].set_xlabel("X Axis")
axes[0].set_ylabel("Y Axis")
```

```
# Second subplot (right)

axes[1].plot(x, y2, color="red", linestyle="--")

axes[1].set_title("Quadratic Growth")

axes[1].set_xlabel("X Axis")


# Prevent overlapping labels/titles

plt.tight_layout()

plt.show()
```

My explanations

I used the code

```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))
```

as a example to show two chart in one block.

Week 1 Activity

Student name: Gordon Tse

Student ID:s223532749

SIT225: Data Capture Technologies

Activity 1.1: Arduino Blink

Welcome to Arduino!

Arduino is an electronic prototyping platform. Different types of sensors & actuators can be attached to Arduino boards to create our own sensing-thinking-acting systems.

Throughout this unit, we will use Arduino to create different sensing devices, and to retrieve the collected sensor data.

In this task, we will try out an introductory exercise, to learn the basic concepts of Arduino.

Hardware Required

Arduino Board with in-built LED

USB cable

Software Required

Arduino programming environment

Pre-requisites: You must do the following before this task

Unit site weekly materials

Active learning sessions require reading material and/or videos available in the unit site, which you must read/view **BEFORE** you start the lab. If you are on-campus, this means that we expect you to have gone through these materials when you join active learning sessions.

Why should you read/watch pre-lab materials?

These materials will help you understand the background which the lab tasks require. Students come to university from diverse backgrounds. Some of you may be familiar with the background information, some of you may not. When you come to the lab prepared, you're already equipped with confidence and will be able to participate in activities better. Ultimately, class time will be much more productive, dynamic, and fun for everyone.

Here are the pre-lab materials for our first task:

1. Watch TED Talk: <https://www.youtube.com/watch?v=UoBUXOOdLXY>
(~15 minutes)
2. Read Arduino tutorial:
<https://www.dummies.com/article/technology/computers/hardware/arduino/how-to-complete-your-first-arduino-sketch-164747>)
3. Read this task sheet from beginning to end.

Task Objective

“We have an Arduino board with an in-built LED light. We need the LED light to be turned on and off continuously, every one second.”

Activity Submission Details

Answer the questions below in this word document and other activities in this activity sheet to create a PDF and submit to OnTrack as described in this week's OnTrack task. PDFs of this activity sheet and OnTrack task need to be merged for submission in the OnTrack portal.

Q1: The TED talk given under the Pre-Lab materials, shows how Arduino is being used for interesting projects to capture data from the environment, process it, and use it to carry out useful actions.

Fill the given table below to answer the following:

What are **three** projects that use captured data as given in the TED talk? What data do they capture? What sensors do you think they could use to capture this data?

Project name	Data captured	Sensors to capture the data
Auto cat feeder	Cat chip distance	Distance sensor or chip detect sensor
Enough	Voice, TV single	Sound sensor, single sensor
Magic clock	Sound, vibration	Sound sensor, vibration sensor

Q2: Consider the given Task Objective. Think about how this simple system can be decomposed to ‘Sense-Think-Act’?

- a) What is the ‘sensing’ requirement in this system, if any?

The sensor require is the LED light.

- b) What is the ‘thinking’ requirement in this system, if any?

The “think” is think is the LED light turn off or on?

- c) What is the ‘acting’ requirement in this system, if any?

The acting is turn on or off the LED Light.

Q3: Please refer to the provided ‘Arduino Blink Activity Sheet’ and follow the steps.

- a) In Arduino-speak, what is a “sketch”?

In Arduino, Sketch mean the code or the instruction I put in IDE.

- b) setup() and loop() are key Arduino constructs. These are required in every Arduino sketch.

- i) Which of the above two, runs once at the very beginning of your program and never again (unless you reset or upload new code)?

Setup()

- ii) Which of the above two, is used to continuously run code over and over again?

Loop()

c) What does **pinMode()** do?

Hint: <http://arduino.cc/en/Reference/HomePage>

pinMode() do set different pin in the Arduino to different mode such as input, output. So it read or output data.

d) What is a comment?

To make a comment do "//", after comment it will be ignore by the computer.

e) What does the following line of code do:

```
delay(x);
```

Hint: <http://arduino.cc/en/Reference/HomePage>

It will stop program running for X sec.

f) There is something you need to check before uploading your sketch. What is this?

Code error and COM port.

Q4: How can you test the Blink program to make sure it is working as given in the Task Objective?

First click the verify to check any error, if no error check the COM port is it connected correctly. If yes then click upload. Now we can see the LED is working.

Activity 1.2: Write Arduino data to serial communication port

Now you can blink Arduino’s built-in LED, it is time to talk to outside Arduino-world, your computer which connects the Arduino board using a USB cable. Arduino IDE shows what you write to the serial port.

Hardware Required

- Arduino Board with in-built LED
- USB cable

Software Required

- Arduino programming environment

Steps:

Steps	Actions
1	Identify the port through which Arduino is connected to your computer. You can find it in Arduino IDE Tools menu. Write Arduino sketch (or download from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_1/sketch_blink.ino), which look like below, in Arduino IDE, deploy in your board and observe the output in IDE’s serial monitor.

```
1 int led_high = 0; // variable to flag ON (1) or OFF (0)
2 int slp_sec = 0; // variable holds number of seconds to sleep
3
4
5 void setup() {
6     pinMode(LED_BUILTIN, OUTPUT); // set LED pin as output
7     digitalWrite(LED_BUILTIN, LOW); // switch off LED pin
8
9     // Baud rate (speed of communication, symbol transfer rate)
10    Serial.begin(6900);
11 }
12
13 void loop() {
14     // sleep randomly selected 1 to 5 seconds
15     slp_sec = random(1, 5);
16
17     if (led_high == 0) {
18         // Write sleeping duration to serial monitor
19         Serial.println("1 -> " + String(slp_sec) + " sec");
20         // Write HIGH to LED
21         digitalWrite(LED_BUILTIN, HIGH);
22     } else {
23         // Write sleeping duration to serial monitor
24         Serial.println("0 -> " + String(slp_sec) + " sec");
25         // Write LOW to LED
26         digitalWrite(LED_BUILTIN, LOW);
27     }
28
29     // Switch LOW to HIGH
30     led_high = (led_high + 1) % 2;
31
32     // Sleep for a while
33     delay(1000*slp_sec);
34 }
35
36
```

Question: Screenshot the serial monitor output and paste the image here.

Answer:

	 <pre>sketch_jul7a.ino 14 15 // Baud rate (speed of communication, symbol transfer rate) 16 Serial.begin(6900); 17 } 18 19 void loop() { 20 // sleep randomly selected 1 to 5 seconds 21 slp_sec = random(1, 5); 22 23 if (led_high == 0) { 24 // Write sleeping duration to serial monitor 25 Serial.println("1 -> " + String(slp_sec) + " sec"); 26 // Write HIGH to LED 27 digitalWrite(LED_BUILTIN, HIGH); 28 29 } else { 30 // Write sleeping duration to serial monitor 31 Serial.println("0 -> " + String(slp_sec) + " sec"); 32 // Write LOW to LED 33 digitalWrite(LED_BUILTIN, LOW); 34 } 35 36 // Switch LOW to HIGH 37 led_high = (led_high + 1) % 2; 38 39 // Sleep for a while</pre> Output Serial Monitor X Message (Enter to send message to 'Arduino NANO 33 IoT' on 'COM3') <pre>1 -> 3 sec 0 -> 3 sec 1 -> 2 sec 0 -> 1 sec 1 -> 2 sec 0 -> 4 sec 1 -> 1 sec 0 -> 1 sec 1 -> 1 sec</pre>
2	Question: Observe the use of “Serial” such as functions Serial.begin() in setup and Serial.println() in loop. Describe what these functions are doing with respect to the serial monitor output you have attached above.

	Answer: the above monitor output is how long it took to turn on and off the LED light. The Serial begin is to open up a communication channel between Arduino board and computer. The number mean Baud Rate, which is the speed of the communication. Serial.println sends text from the Arduino back to computer screen so I have output to see.
3	Question: If Arduino transfers data at 4800 bits per second (baud rate) and you're sending 12 bytes of data, how long does it take to send over this information? Answer: 1 byte = 8 bits 12 byte * 8 = 96 bits 96 bits / 4800bits = 0.02s

Activity 1.3: Arduino talks to Python

To listen to what Arduino sends, there will be a Python program running and keep listening to the same port where Arduino is writing data to receive it.

Hardware Required

Arduino Board with in-built LED

USB cable

Software Required

Arduino programming environment

Python 3.0 (Follow Python installation manual in unit site)

Steps:

Steps	Actions
1	Write Arduino sketch (or download from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_1/sketch_serial_comm.ino) which looks like below. Open in Arduino IDE. Study the code. Upload the code to Arduino board and observe output in Arduino IDE serial monitor.

```
1
2 int x;
3
4 void setup() {
5     Serial.begin(9600); // set baud rate
6 }
7
8 void loop() {
9     while (!Serial.available()) {} // wait for data to arrive
10
11     // read string data from Serial, we know Python
12     // script is sending just an integer.
13     x = Serial.readString().toInt();
14
15     // write a string (no newline)
16     Serial.print("Arduino sends: ");
17
18     // Add 1 to what received.
19     // Write an integer with a newline (println vs print).
20     Serial.println(x + 1);
21
22     // Push the data through serial channel.
23     Serial.flush();
24 }
```

Question: Do you see any output in serial monitor? If not, then why?

Answer: this code is waiting user to input a number then Arduino turn input to int and do input + 1 then output. So it will appear any output at first. It will output only if I input a number.

- 2 Write Python code as below (or download from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_1/serial_comm_script.py) and save it to a file serial_comm_script.py.

```
1 import serial
2 import random
3
4 # set baud rate, same speed as set in your Arduino sketch.
5 boud_rate = 9600
6
7 # set serial port as suits your operating system
8 s = serial.Serial('/dev/ttyACM1', boud_rate, timeout=5)
9
10 while True: # infinite loop, keep running
11
12     # a random number between 5 and 50.
13     data_send = random.randint(5, 50)
14
15     # write to serial port, set data encoding.
16     # Raw bytes are sent through serial ports, Python bytes() needs
17     # to know the encoding to generate bytes from string.
18     #
19     # We send a single integer which is read from Arduino sketch.
20     #
21     d = s.write(bytes(str(data_send), 'utf-8'))
22     print(f"Send >>> {data_send} ({d} bytes)")
23
24     # Read from serial port.
25     #
26     # readline keeps reading until a newline found in the data stream.
27     # Unlike write above, we just send an integer with no newline.
28     # You should receive data the same way as it is sent.
29     #
30     d = s.readline().decode("utf-8")
31     print(f"Recv <<< {d}")
32
```

```

serial_comm_script.py X
C: > Users > gordo > OneDrive > 桌面 > Deakin > Y3 > T2 > SIT225 Data Capture > Week 1 > serial_comm_script.py > ...
14 # s = serial.Serial('/dev/ttyACM1', boud_rate, timeout=5)
15 s = serial.Serial('COM3', boud_rate, timeout=5)
16
17 while True: # infinite loop, keep running
18     # a random number between 5 and 50.
19     data_send = random.randint(5, 50)
20
21     # write to serial port, set data encoding.
22     # Raw bytes are sent through serial ports, Python bytes() needs
23     # to know the encoding to generate bytes from string.
24     #
25     # We send a single integer which is read from Arduino sketch.
26     #
27     d = s.write(bytes(str(data_send), 'utf-8'))
28     print(f"Send >>> {data_send} ({d} bytes)")
29
30
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR bash - Week 1
gordo@MSI MINGW64 /c/Users/gordo/OneDrive/桌面/Deakin/Y3/T2/SIT225 Data Capture/Week 1
$ py serial_comm_script.py
KeyboardInterrupt

gordo@MSI MINGW64 /c/Users/gordo/OneDrive/桌面/Deakin/Y3/T2/SIT225 Data Capture/Week 1
$ py serial_comm_script.py
Traceback (most recent call last):
  File "C:\Users\gordo\OneDrive\桌面\Deakin\Y3\T2\SIT225 Data Capture\Week 1\serial_comm_script.py", line 15, in <module>
    s = serial.Serial('COM3', boud_rate, timeout=5)
  File "C:\Users\gordo\AppData\Local\Programs\Python\Python312\Lib\site-packages\serial\serialwin32.py", line 33, in __init__
    super(Serial, self).__init__(*args, **kwargs)
  File "C:\Users\gordo\AppData\Local\Programs\Python\Python312\Lib\site-packages\serial\serialutil.py", line 244, in __init__
    self.open()
  File "C:\Users\gordo\AppData\Local\Programs\Python\Python312\Lib\site-packages\serial\serialwin32.py", line 64, in open
    raise SerialException("could not open port {!r}: {!r}".format(self.portstr, ctypes.WinError()))
serial.serialutil.SerialException: could not open port 'COM3': PermissionError(13, '存取被拒', None, 5)

gordo@MSI MINGW64 /c/Users/gordo/OneDrive/桌面/Deakin/Y3/T2/SIT225 Data Capture/Week 1
$

```

Run the Python file from command line using command:

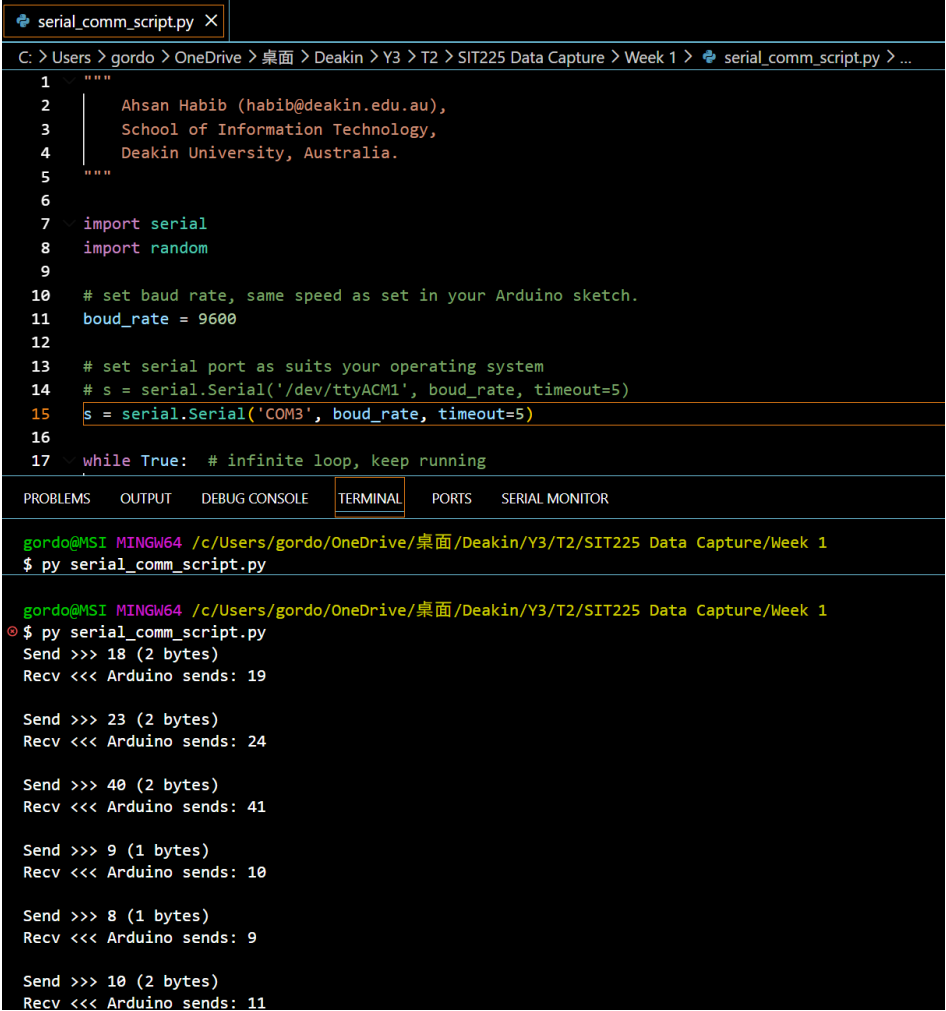
\$ python serial_comm_script.py

Question: Run Python script in command line, does the script run or do you receive any error? If there is an error, analyse the error message and identify what went wrong.

Answer: “ModuleNotFoundError: No module named 'serial'” I receive this error, this appear because my python don’t have a way to communicate with the Arduino not like Arduino IDE it have a built in function that can connect with the board. If I want to use python addition lib need to install.

After install the py -m pip install pyserial I get error ‘could not open port '/dev/ttyACM1' this is because it not match my OS system so I change to COM3.

When I opening IDE, the py file will appear error “could not open port 'COM3'.

	This is because IDE is using or talking the board.
3	Question: Following the above step 2, do you think Arduino IDE is keeping the serial communication port busy talking to Arduino board? Now try to close the Arduino IDE and repeat step 2 above to run the Python script. Do previous errors show up again? If not, Python script should have print messages in the command line. Take the screenshot of the Python script output and paste here. Answer: “could not open port 'COM3' that the error I get when I open last IDE and try to run py file at the same time. So yes Arduino IDE is keeping serial communication port busy talking to Arduino board. After I close IDE, py file work.  <pre> serial_comm_script.py X C: > Users > gordo > OneDrive > 桌面 > Deakin > Y3 > T2 > SIT225 Data Capture > Week 1 > serial_comm_script.py > ... 1 """ 2 Ahsan Habib (habib@deakin.edu.au), 3 School of Information Technology, 4 Deakin University, Australia. 5 """ 6 7 import serial 8 import random 9 10 # set baud rate, same speed as set in your Arduino sketch. 11 boud_rate = 9600 12 13 # set serial port as suits your operating system 14 # s = serial.Serial('/dev/ttyACM1', boud_rate, timeout=5) 15 s = serial.Serial('COM3', boud_rate, timeout=5) 16 17 while True: # infinite loop, keep running PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR gordo@MSI MINGW64 /c/Users/gordo/OneDrive/桌面/Deakin/Y3/T2/SIT225 Data Capture/Week 1 \$ py serial_comm_script.py gordo@MSI MINGW64 /c/Users/gordo/OneDrive/桌面/Deakin/Y3/T2/SIT225 Data Capture/Week 1 \$ py serial_comm_script.py Send >>> 18 (2 bytes) Recv <<< Arduino sends: 19 Send >>> 23 (2 bytes) Recv <<< Arduino sends: 24 Send >>> 40 (2 bytes) Recv <<< Arduino sends: 41 Send >>> 9 (1 bytes) Recv <<< Arduino sends: 10 Send >>> 8 (1 bytes) Recv <<< Arduino sends: 9 Send >>> 10 (2 bytes) Recv <<< Arduino sends: 11 </pre>
4	Question: Observe the Python script output and describe the communication protocol used between Arduino sketch and Python script. Answer: looking at the code and output, this program first generate a random

	number and send to Arduino and let Arduino do $x+1$ math and send back the result and print out result. The port code in py code is a bit different because it use different system then IDE. /dev/ttyACM1 VS 'COM3'. They use same boud_rate .
--	--

Student name: Gordon Tse

Student ID: s223532749

SIT225: Data Capture Technologies

Activity 2.1: Working with sensor - DHT22

DHT22 is a temperature and humidity sensor.

Hardware Required

Arduino Board

DHT22 sensor

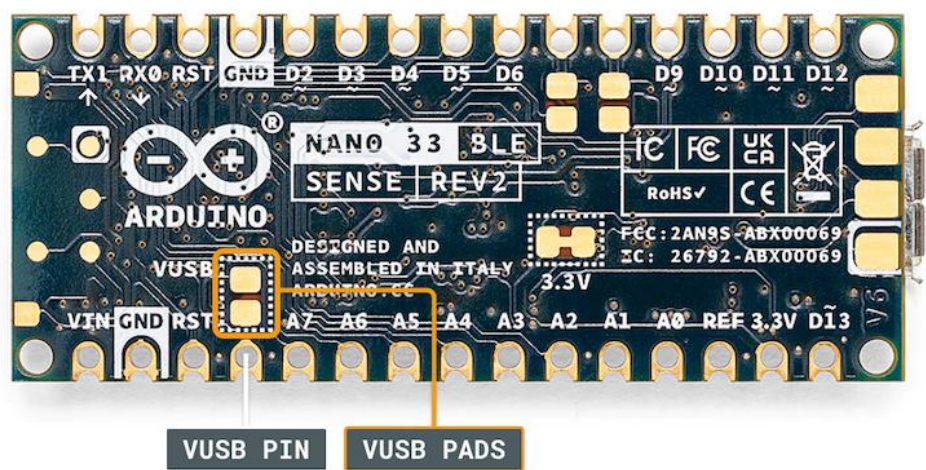
USB cable

Software Required

Arduino programming environment

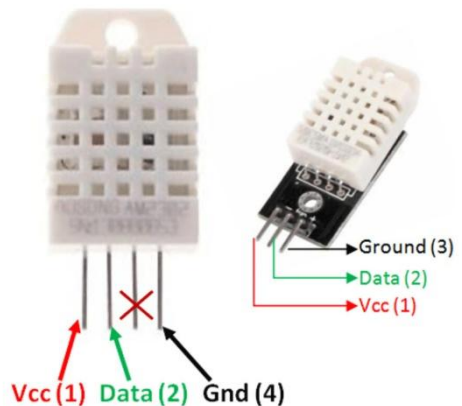
Known issue, action required

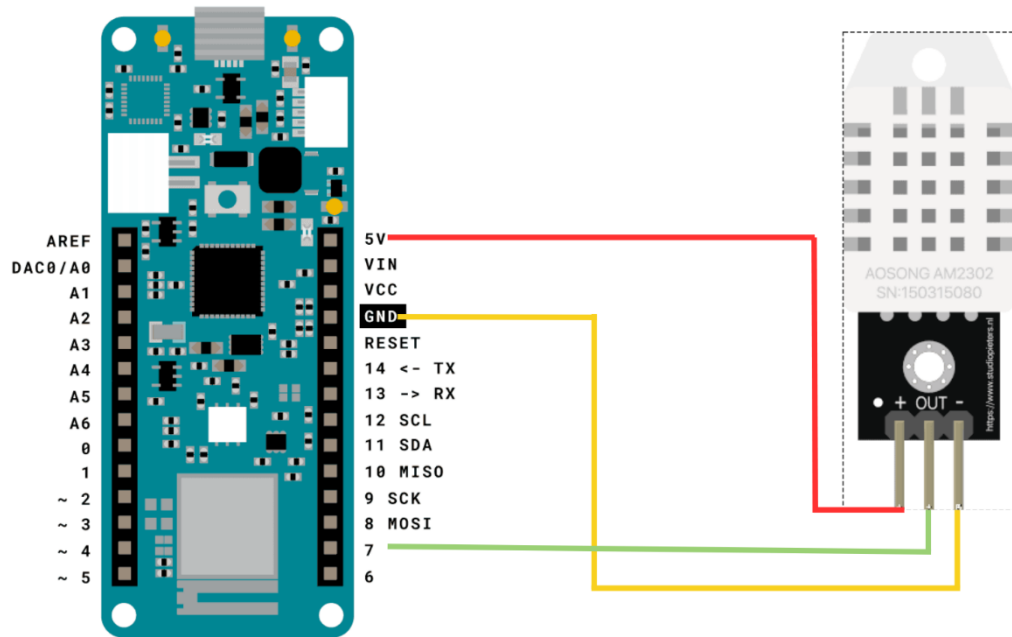
Arduino Nano 33 IoT board operates on 3.3 V, it needs to be arranged to make it 5 V. The Arduino board has a pin called VUSB or VBUS and there are 2 pads next to the pin. **These two pads must be shorted to enable the pin** (see detail here <https://support.arduino.cc/hc/en-us/articles/360014779679-Enable-5-V-power-on-the-VUSB-or-VBUS-pin-on-Nano-boards>).



To test, you can use a wire and connect these 2 pads manually by hand to see if data is coming through DHT22 sensor. For long data collection, you will need to solder the wire permanently to connect the pads. Seeking help from tutors there is an on-campus facility called Maker Space where you can do it.

Steps:

Step	Action
1	Connect your DHT22 Temperature and Humidity Sensor to the Arduino board. Note that the pin layout in the image below may look different than the board you may have.  The diagram shows two views of a DHT22 sensor. The left view shows the sensor with four pins labeled Vcc (1), Data (2), and Gnd (4). The right view shows the sensor with three pins labeled Ground (3), Data (2), and Vcc (1). A red 'X' is placed over the Data pin in the left view, indicating it should not be connected to the Data pin of the Arduino board.



- Pick a red male-female jumper wire and attach the female end to pin 1 (VCC pin) on the sensor. Plug the male end into the Arduino board's 5V power pin.
- Pick a blue male-female jumper wire and attach the female end to pin 2 (DATA pin) on the sensor. Plug the male end into the Arduino board's digital data pin 2.
- Pick a black male-female jumper wire and attach the female end to pin 4 (GND) on the sensor. Plug the male end into the Arduino board's GND pin.
- Sensor's pin 3 is not used.

2 Connect your Arduino board to your computer using the USB cable.

3 Write an Arduino sketch (or download it from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_dht22.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

	<pre> 6 7 // Add "DHT sensor library" 8 #include <DHT.h> 9 10 #define DHTPIN 2 // digital pin number 11 #define DHTTYPE DHT22 // DHT type 11 or 22 12 DHT dht(DHTPIN, DHTTYPE); 13 14 // variables to store data. 15 float hum, temp; 16 17 void setup() { 18 // Set baud rate for serial communication 19 Serial.begin(9600); 20 21 // initialise DHT library 22 dht.begin(); 23 } 24 25 void loop() { 26 // read data 27 hum = dht.readHumidity(); 28 temp = dht.readTemperature(); 29 30 // Print data to serial port - a long way 31 // 32 // Serial.print("Humid: "); 33 // Serial.print(hum); 34 // Serial.print(" %, Temp: "); 35 // Serial.print(temp); 36 // Serial.println(" Celsius"); 37 38 // Print data to serial port - a compact way 39 Serial.println(String(hum) + "," + String(temp)); 40 41 // wait a while 42 delay(15*1000); 43 } </pre>
4	Question: A spec of the DHT22 sensor is given in the link below. It mentions that the sampling rate is 0.5 Hz. https://lastminuteengineers.com/dht11-dht22-arduino-tutorial i) What does the sampling rate mean? ii) Where is this used in the Arduino code? Answer: Sample rate mean how many times per sec the device measure real-world signal. In the line 42 “delay (15*1000)”.
5	Question: Take a screenshot of your Serial Monitor displaying temperature & humidity sensor data logs. Add the image here. Answer:

→ ↻

📶 Arduino NANO 33 IoT

code.ino

```
9  #include <DHT.h>
10
11  #define DHTPIN 21 // digital pin number
12  #define DHTTYPE DHT22 // DHT type 11 or 22
13  DHT dht(DHTPIN, DHTTYPE);
14
15  // variables to store data.
16  float hum, temp;
17
18  void setup() {
19      // Set baud rate for serial communication
20      Serial.begin(9600);
21
22      // initialise DHT library
23      dht.begin();
24  }
25
26  void loop() {
27      // read data
28      hum = dht.readHumidity();
29      temp = dht.readTemperature();
30
31      // Print data to serial port - a long way
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino NANO 33 IoT' on 'COM3')

62.50,18.00
62.80,17.90
63.20,17.90
63.60,17.90
63.80,17.90
64.10,17.80
64.00,17.80
63.20,17.80
62.90,17.80
63.00,17.80
62.80,17.80
62.50,17.80

Week 2 Activity

Activity 2.2: Working with sensor - HC-SR04

HC-SR04 is an Ultrasonic sensor.

Hardware Required

Arduino Board

HC-SR04 Ultrasonic sensor

USB cable

Software Required

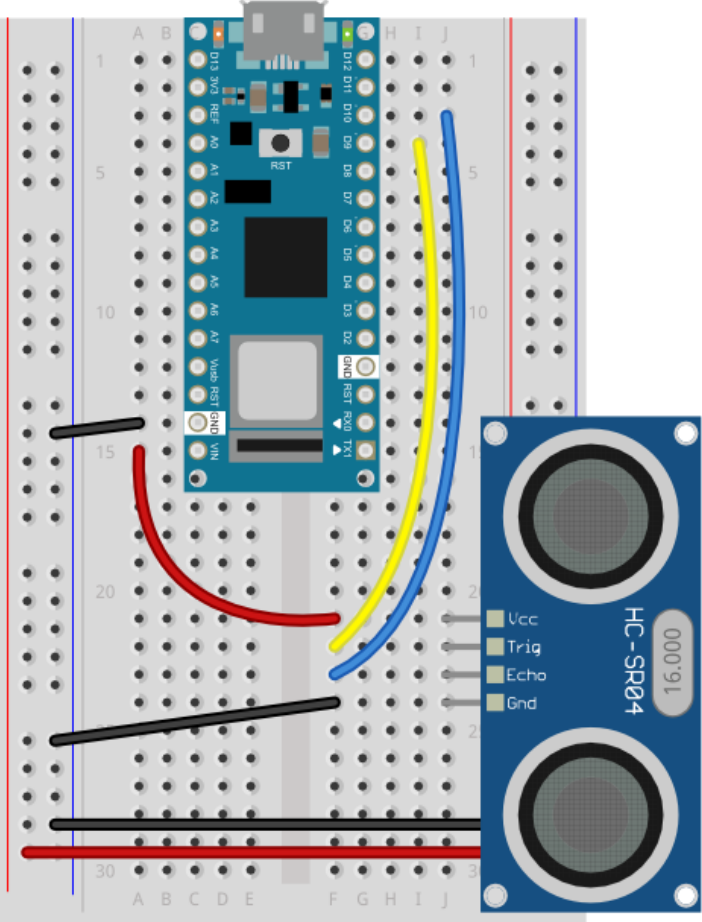
Arduino programming environment

Known issue, action required

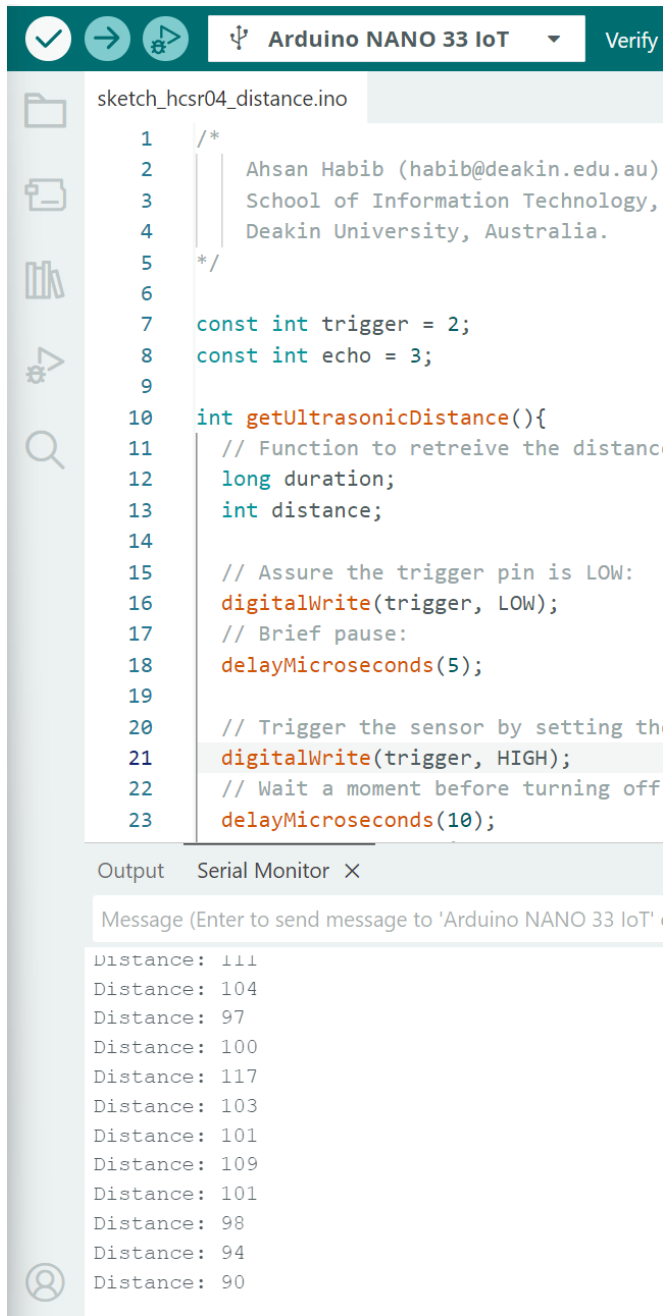
The same known issue applies to SR04 which operates at 5 V source while the Arduino Nano 33 IoT supplies 3.3 V. It requires 2 VUSB (or VBUS) pads to be shorted to enable the pin.

Steps:

Step	Action
1	Connect your HC-SR04 sensor to the Arduino board. Note that the pin layout in the image below may look different than the board you may have.

	
2	Connect your Arduino board to your computer using the USB cable.
3	Write an Arduino sketch (or download it from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_hcsr04_distance.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

	<pre> 7 const int trigger = 2; 8 const int echo = 3; 9 10 int getUltrasonicDistance(){ 11 // Function to retrieve the distance reading of the ultrasonic 12 long duration; 13 int distance; 14 15 // Assure the trigger pin is LOW: 16 digitalWrite(trigger, LOW); 17 // Brief pause: 18 delayMicroseconds(5); 19 20 // Trigger the sensor by setting the trigger to HIGH: 21 digitalWrite(trigger, HIGH); 22 // Wait a moment before turning off the trigger: 23 delayMicroseconds(10); 24 // Turn off the trigger: 25 digitalWrite(trigger, LOW); 26 27 // Read the echo pin: 28 duration = pulseIn(echo, HIGH); 29 // Calculate the distance in centimeter (CM): 30 distance = duration * 0.034 / 2; 31 32 // Uncomment this line to return value in IN instead of CM: 33 //distance = distance * 0.3937008 34 35 // Return the distance read from the sensor: 36 return distance; 37 } 38 39 void setup() { 40 // Define inputs and outputs: 41 pinMode(trigger, OUTPUT); 42 pinMode(echo, INPUT); 43 44 // Start the serial monitor: 45 Serial.begin(9600); 46 } 47 48 void loop() { 49 // Print the distance to the serial monitor: 50 Serial.print("Distance: "); 51 Serial.println(getUltrasonicDistance()); 52 53 // Wait one second before continuing: 54 delay(1000); 55 } </pre>
4	Question: Spec of SR04 is available here (https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf). Identify 2 critical aspects you should be careful about this sensor operation. Answer: 1. This sensor might not work with surfaces that will absorb the sound wave, also might not work with mirror so need to be careful what object we want to detect. 2. if object getting to close it might not accurate.

	3. and it need time to process to get back result.
5	Question: Take a screenshot of your Serial Monitor displaying distance values while you try to generate a periodic motion by moving your hand gradually back and forth towards the sensor. Add the image here. Answer:  The screenshot shows the Arduino IDE interface. The top bar indicates the board is 'Arduino NANO 33 IoT'. The sketch file is 'sketch_hcsr04_distance.ino'. The code in the editor is as follows: <pre>1 /* 2 Ahsan Habib (habib@deakin.edu.au) 3 School of Information Technology, 4 Deakin University, Australia. 5 */ 6 7 const int trigger = 2; 8 const int echo = 3; 9 10 int getUltrasonicDistance(){ 11 // Function to retrieve the distance 12 long duration; 13 int distance; 14 15 // Assure the trigger pin is LOW: 16 digitalWrite(trigger, LOW); 17 // Brief pause: 18 delayMicroseconds(5); 19 20 // Trigger the sensor by setting the 21 digitalWrite(trigger, HIGH); 22 // Wait a moment before turning off 23 delayMicroseconds(10);</pre> The Serial Monitor is open, showing the following output: <pre>Distance: 111 Distance: 104 Distance: 97 Distance: 100 Distance: 117 Distance: 103 Distance: 101 Distance: 109 Distance: 101 Distance: 98 Distance: 94 Distance: 90</pre>

Activity 2.3: Working with sensor - Accelerometer

LSM6DS3 module on the Arduino Nano 33 IoT is an accelerometer and gyroscope sensor.

Hardware Required

Arduino Nano 33 IoT Board (has inbuilt LSM6DS3 module)

USB cable

Software Required

Arduino programming environment

Steps:

Step	Action
1	Write Arduino sketch (or download code from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_accelero.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

	<pre> 6 7 // Add Arduino_LSM6DS3 library 8 // from Arduino IDE Library Manager. 9 #include <Arduino_LSM6DS3.h> 10 11 float x, y, z; 12 13 void setup() { 14 Serial.begin(9600); // set baud rate 15 while (!Serial); // wait for port to init 16 Serial.println("Started"); 17 18 if (!IMU.begin()) { 19 Serial.println("Failed to initialize IMU!"); 20 while (1); 21 } 22 23 Serial.println(24 "Accelerometer sample rate = " 25 + String(IMU.accelerationSampleRate()) + " Hz"); 26 } 27 28 void loop() { 29 // read accelero data 30 if (IMU.accelerationAvailable()) { 31 IMU.readAcceleration(x, y, z); 32 } 33 34 Serial.println(35 String(x) + ", " + String(y) + ", " + String(z)); 36 37 delay(1000); 38 } </pre>
2	Question: Spec of LSM6DS3 is available here (https://content.arduino.cc/assets/st_imu_lsm6ds3_datasheet.pdf). Identify at least 3 attributes of this sensor you think important to work with. Answer: 1. Unlike simpler sensors that require the Arduino to constantly ask for data, the LSM6DS3 has a built-in 8 Kilobyte FIFO memory buffer. So it more efficient. 2. it can do sensitivity tuning which it can adjust to suitable sensitivity for different project. 3. it have both accelerometer and gyroscope
3	Question: Take a screenshot of your Serial Monitor displaying sensor readings. Add the image here.

Answer:

sketch_accelero.ino

```

1  /*
2     Ahsan Habib (habib@deakin.edu.au),
3     School of Information Technology,
4     Deakin University, Australia.
5  */
6
7  // Add Arduino_LSM6DS3 library
8  // from Arduino IDE Library Manager.
9  #include <Arduino_LSM6DS3.h>
10
11 float x, y, z;
12
13 void setup() {
14     Serial.begin(9600); // set baud rate
15     while (!Serial); // wait for port to init
16     Serial.println("Started");
17
18     if (!IMU.begin()) {
19         Serial.println("Failed to initialize IMU!");
20         while (1);
21     }
22
23     Serial.println(

```

Output Serial Monitor X

Message (Enter to send message to 'Arduino NANO 33 IoT' on 'COM3')

```

0.46, -0.41, 0.78
0.44, -0.52, 0.78
0.03, -0.87, 0.59
-0.12, -0.77, 0.62
-0.12, -0.81, 0.62
-0.16, -0.62, 0.77
-0.06, 0.66, 0.78
-0.10, 0.71, 0.66
-0.25, 0.67, 0.66
-0.33, 0.65, 0.78
-0.23, 0.52, 0.90
0.07, 0.00, 1.00

```

4

Question: Identify the max sampling rate and consider reducing the delay (line 37 in the sketch) to increase the number of samples. Summarise your findings here.

Answer: Accelerometer sample rate = 104.00 Hz in code it showing 104 HZ so 104 times per second is the fastest sample rate. For now delay(1000) is printing out 1 per sec. by removing this line, it will output as fast as it can that is 104 per sec.

Activity 2.4: Plot data using Python Notebook

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. You can find detail in official website (<https://matplotlib.org>).

Hardware Required

No hardware required

Software Required

Python Jupyter notebook

Steps:

Step	Action
1	Download the Jupyter notebook week2_notebook.ipynb from here (https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/week2_notebook.ipynb). Follow the instructions in the notebook to carry out instructions and finally convert the notebook to PDF so you can combine it with this activity sheet PDF.

```
student_name = "Gordon" # fill your name
student_id = "s223532749" # fill your student ID
print("Student name: " + student_name)
print("Student ID: " + student_id)
```

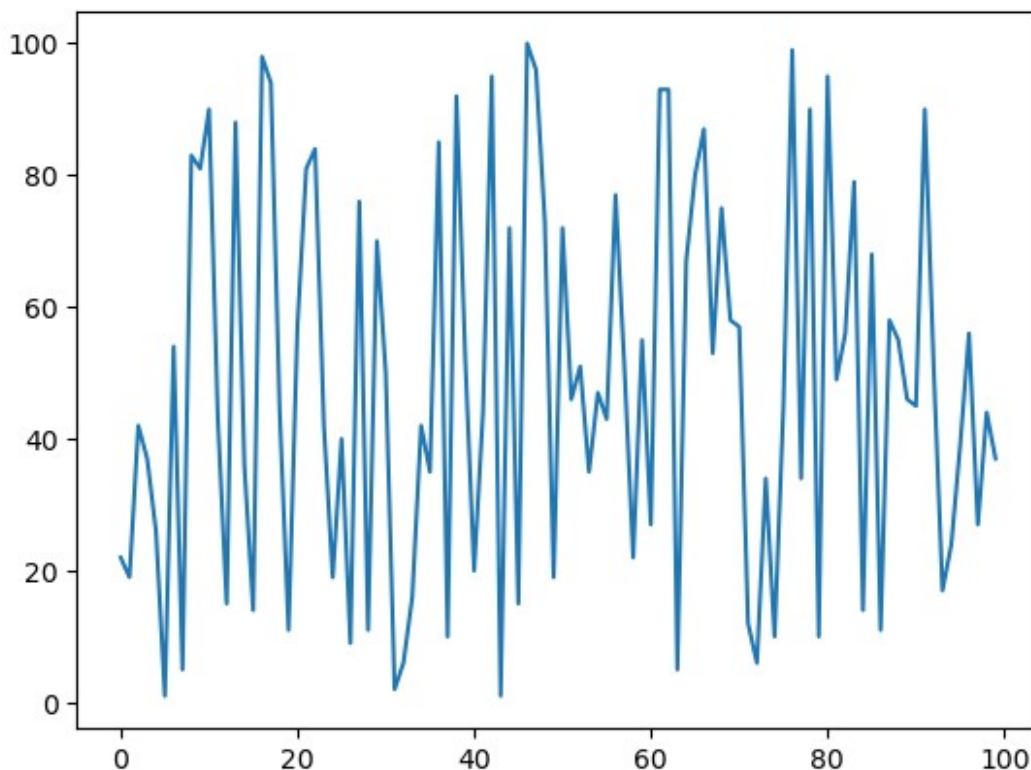
Student name: Gordon
Student ID: s223532749

```
import random
import matplotlib.pyplot as plt
```

```
n_values = 100
y_values = []
```

```
# Create data (y_values) randomly between 1 and 100.
for i in range(n_values):
    y_values.append(random.randint(1, 100))
```

```
x_values = range(n_values) # X is sequence of values 0-99
plt.plot(x_values, y_values)
plt.show()
```



```
# Plot 2 variables
#
```

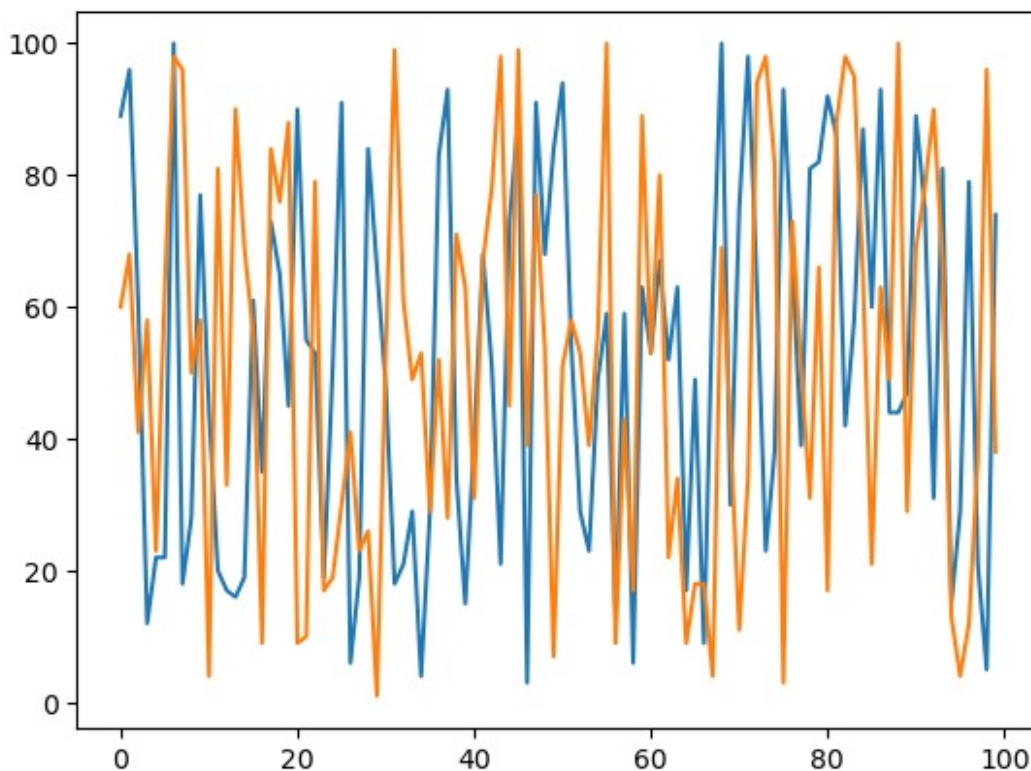
```

n_values = 100
y_values_1 = []
y_values_2 = []

# Create data (y_values) randomly between 1 and 100.
for i in range(n_values):
    y_values_1.append(random.randint(1, 100))
    y_values_2.append(random.randint(1, 100))

x_values = range(n_values) # X is sequence of values 0-99
plt.plot(x_values, y_values_1)
plt.plot(x_values, y_values_2) # call plot again draws in the same
graph.
plt.show()

```

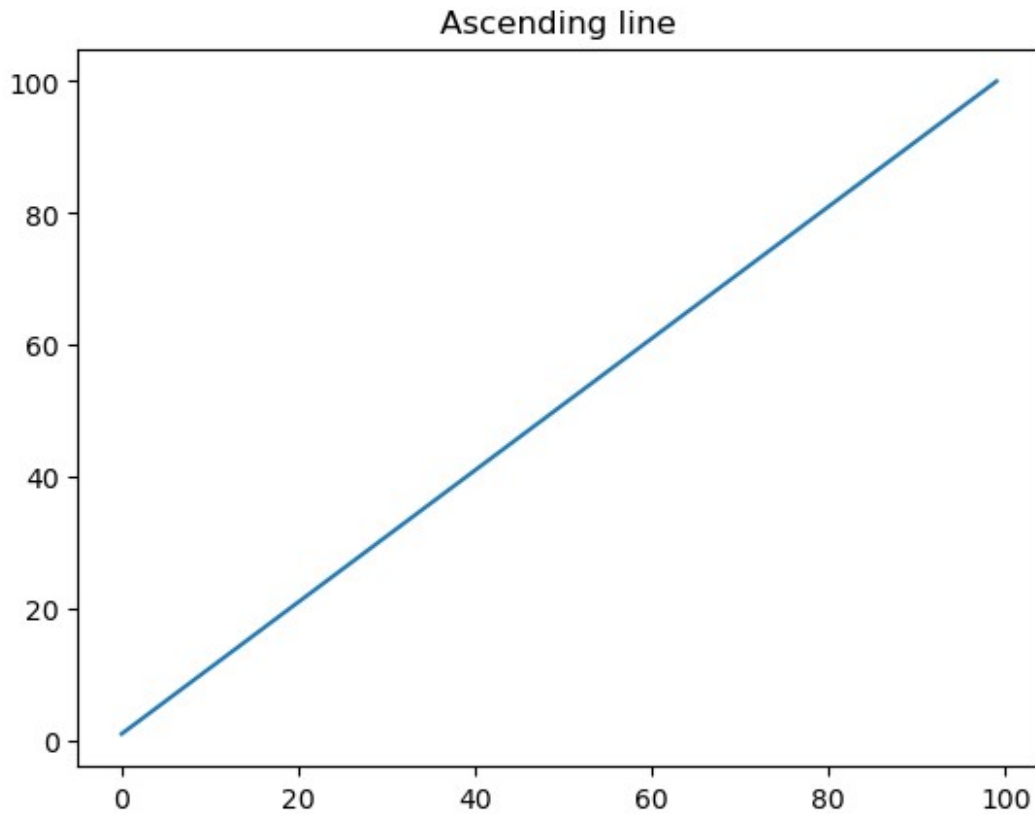


```

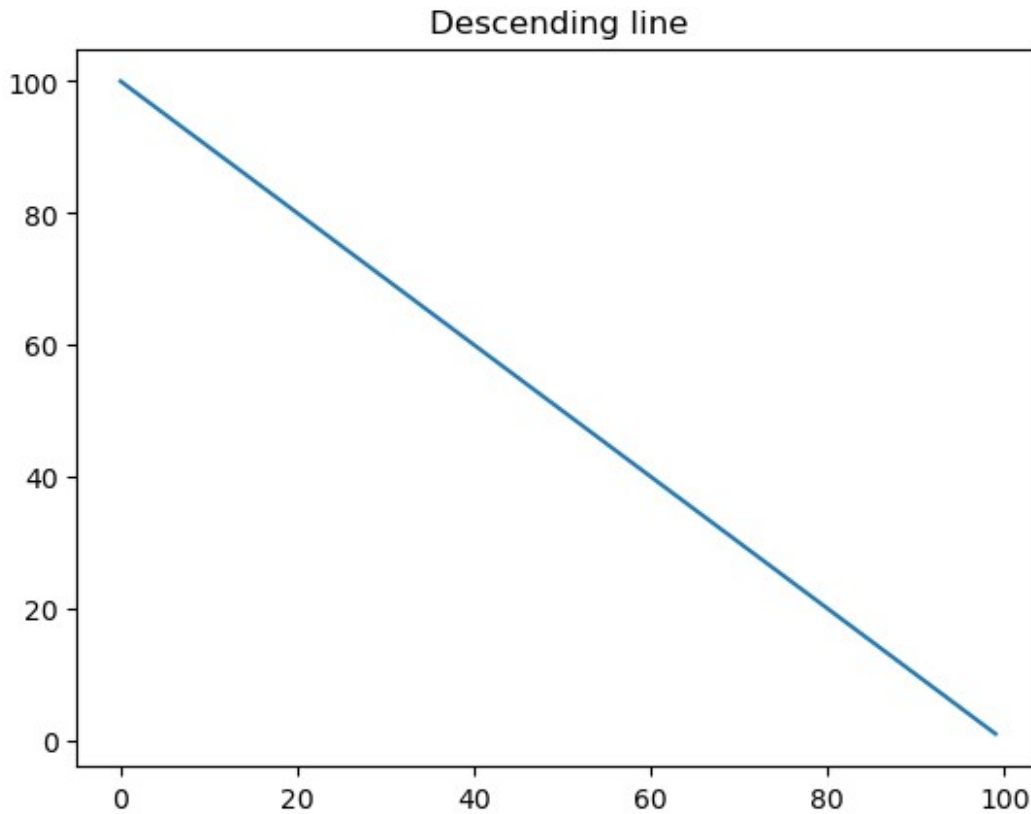
#
# Activity 1: Create data so that the plot draws an
# ascending line (y_values increase at any rate).
#

y_values_asc = list(range(1, n_values + 1))
plt.plot(x_values, y_values_asc)
plt.title("Ascending line")
plt.show()

```



```
#  
# Activity 2: Create data so that the plot draws a  
# descending line (y_values decrease at any rate).  
#  
y_values_desc = list(range(n_values, 0, -1))  
plt.plot(x_values, y_values_desc)  
plt.title("Descending line")  
plt.show()
```



```
#  
# Activity 3: Create data so that the plot draws a  
# wave. You can consider using Python's math library, which has  
# a sin function (detail  
https://www.w3schools.com/python/ref\_math\_sin.asp).  
#  
import math  
  
amplitude = 50  
cycles = 3 # number of full sine cycles across the x range  
y_wave = [amplitude * math.sin(2 * math.pi * cycles * (i / n_values))  
for i in x_values]  
  
plt.plot(x_values, y_wave)  
plt.title("Sine wave")  
plt.show()
```

Sine wave

